

Ghost Chains

Event and Scoring

This challenge is released in **three cumulative phases**.

Current Phase	Evaluation Includes
Phase 1	Phase 1 requirements
Phase 2	Phases 1 and 2 requirements
Phase 3	Phases 1, 2 and 3 requirements

Each phase's evaluation re-tests the requirements of every earlier phase within the same evaluation run. A Phase 2 evaluation therefore re-checks Phase 1 behaviour, and a Phase 3 evaluation re-checks Phases 1 and 2 behaviour. Extend your system without breaking what already works.

Phases Schedule

Start with Phase 1 only. Phases 2 and 3 are unlocked during the event — ignore them until announced. When they arrive, extend your system without breaking what already works.

Scoring Overview

You do **not** need to match an absolute “correct” score for each transaction. You need a coherent model that consistently ranks suspicious activity above ordinary flow.

Submissions are evaluated across two dimensions:

- **Detection Quality:** How accurately your system ranks more suspicious transactions above less suspicious ones.
- **Structural Consistency:** How coherently your model behaves across structurally related scenarios. Systems built on principled graph models are expected to outperform implementations tuned to specific patterns.

In addition, evaluating a phase during its **active window** grants an **earliness bonus**. The bonus rewards a working system evaluated early within a phase's window; it is applied on top of the two scored dimensions and does not change what is scored.

Final standing reflects the two scored dimensions combined, with the per-phase earliness bonus applied.

Phase start times are announced via the coordinator and the challenge room / Teams channel; they are not published in advance in these documents.

System Requirements

Service Overview

Participants must implement a real-time **transaction risk scoring service** for an anti-money laundering (AML) environment.

The system receives transactions continuously and must assign a **risk score between 0.0 and 1.0** to each transaction at the moment it is processed.

The service is expected to operate under **stateful, streaming conditions**, where historical transactions within the active lookback window influence future scoring behavior.

Required Endpoints

Participants must implement the following HTTPS endpoints:

Health Check

```
GET /ghost-chains/health
```

```
{ "status": "ok" }
```

This endpoint is used to verify service availability.

State Reset

```
POST /ghost-chains/reset
```

Request:

```
{ "clearTransactions": true }
```

Response:

```
{ "clearTransactions": true }
```

Behavior:

- Clears all internal state related to previously processed transactions.
- Resets graph construction, caches, and any derived structures.
- Must restore the system to a clean initial state equivalent to startup.

Transaction Processing

POST /ghost-chains/transactions

Request fields:

- transactions : Array of transaction objects:
 - txId : Unique string identifier for the transaction.
 - fromUserId : Identifier of the sending entity. (User is a convenience label for any identity — account, legal entity, or other counterparty.)
 - toUserId : Identifier of the receiving entity.
 - amount : Transfer amount as a number.
 - createdAt : ISO 8601 timestamp of the transaction.
 - ipAddress (*optional*): Network address used to initiate the transaction. Omitted when unknown.
 - deviceId (*optional*): Device identifier used to initiate the transaction. Omitted when unknown.

Optional fields may be absent on any transaction; this must not cause processing to fail.

Request:

```
{
  "transactions": [
    {
      "txId": "tx_meridian_001",
      "fromUserId": "meridian_holdings",
      "toUserId": "apex_logistics",
      "amount": 370.0,
      "createdAt": "2026-06-08T12:00:00Z"
    },
    {
      "txId": "tx_cascade_014",
      "fromUserId": "cascade_payments",
      "toUserId": "horizon_capital",
      "amount": 100.0,
      "createdAt": "2026-06-08T12:01:00Z"
    }
  ]
}
```

Response fields:

- transactions : Array of result objects:
 - txId : Echoes the transaction identifier from the request.
 - riskScore : Risk score in [0.0, 1.0].

Response:

```
{
  "transactions": [
    { "txId": "tx_meridian_001", "riskScore": 0.0 },
    { "txId": "tx_cascade_014", "riskScore": 0.0 }
  ]
}
```

Behavior:

- Each transaction must be assigned a risk score immediately upon processing.
- Multiple transactions in a single request must be processed sequentially in order.
- The response must preserve input ordering.

Getting Started

1. Implement the three endpoints above.
2. Smoke-test with the commands below. Replace `localhost:8080` with your server's host address.
3. Register your public base URL with the coordinator and trigger an evaluation.

```
curl -s http://localhost:8080/ghost-chains/health
```

```
curl -s -X POST http://localhost:8080/ghost-chains/reset \
-H 'Content-Type: application/json' \
-d '{"clearTransactions": true}'
```

```
curl -s -X POST http://localhost:8080/ghost-chains/transactions \
-H 'Content-Type: application/json' \
-d '{
  "transactions": [
    {
      "txId": "tx_meridian_001",
      "fromUserId": "meridian_holdings",
      "toUserId": "apex_logistics",
      "amount": 370.0,
      "createdAt": "2026-06-08T12:00:00Z"
    }
  ]
}'
```

State and Execution Model

Build a stateful, streaming service that maintains a rolling graph of account transactions and assigns relative risk scores within a bounded lookback window.

Streaming state. Score each transaction using only information available at that point. State is updated incrementally — no reprocessing of history is required. Maintain state sufficient for:

- Graph relationships between entities
- Temporal ordering of transactions
- Derived structural and behavioural signals

Lookback window ($W = 24$ hours). Only transactions created within the most recent 24 hours are active. Expired transactions must be removed from graph state and must not influence scoring. Be precise about boundary conditions.

Ordering. Within a single request, process transactions in the order provided. Across requests, arrival order defines global state evolution.

Idempotency. Each `txId` is unique. If a `txId` is submitted more than once with an identical payload, return the original score and make no state changes. Consider what should happen if the payload differs.

Risk scores are real numbers in `[0.0, 1.0]` representing relative suspiciousness — not calibrated probabilities. A higher score means more suspicious. Scores must be comparable within the same running system state, and for identical inputs after a reset, outputs must be consistent.

Forward compatibility. Later phases introduce new optional fields. Ignore unknown or absent fields gracefully; do not reject transactions that include unrecognised attributes.

Performance. Handle continuous streaming input with memory usage bounded by the active lookback window.

System Observation and Diagnostics

When your submission produces a ranking that disagrees with the reference model, the platform does not disclose absolute scores or internal evaluation details. Instead, a diagnostic array is returned identifying which signal dimensions showed disagreement. A phase's evaluation can emit only the observation categories listed in that phase's document.

Severity Interpretation

Observation categories describe where evaluator disagreement was detected. Multiple categories may be reported simultaneously. Severity reflects the magnitude of disagreement between your submission's output and reference behaviour for the evaluated scenario. Severity is computed dynamically and is not tied to any specific challenge phase or test difficulty level. If no disagreement is detected for a scenario, no diagnostic payload is generated.

Diagnostic Payload Format

`STRUCTURAL_DEVIATION`: Moderate, `TEMPORAL_DEVIATION`: Low

Overview

Criminal networks move billions through ordinary financial systems every year. They use digital assets, real-time payments, capital markets and seemingly legitimate accounts in an almost invisible money trail — flickering with the rhythmic taps on a screen.

As transactions arrive continuously, systems that rely on full historical recomputation cannot keep pace. Risk must be assessed **incrementally** as each transaction arrives, using only prior information.

Your Mission

Build a **real-time risk scoring service** that assigns a risk score to each incoming transaction based on an evolving graph.

The dataset is synthetic but designed to reflect realistic financial behaviour. The goal is to detect coordinated activity as it emerges, while minimizing false positives on ordinary business transactions.

Entity names in the examples are **synthetic** and chosen to resemble counterparties that often appear in trade-based or layering typologies (holding companies, logistics fronts, payment intermediaries, and import/export shells).

Phase 1 - Follow the Money (Structural Signal)

Phase 1 Briefing



In plain terms: watch how money moves between entities. A lonely Meridian Holdings → Apex Logistics transaction is usually boring. Money that travels onward, fans into the same destination, or — especially — **loops back** through entities you have already seen is more interesting.

Your task is to assign a higher risk score to transactions that increase this structural signal.

Core Principle

Each incoming transaction updates a directed graph of entities.

Risk score reflects how the transaction changes the graph's structural signal: the combined effect of new or shortened paths between entities, not any single graph feature. A higher risk score corresponds to a greater increase in the graph's capacity to support recurring flow. Edge cases the examples do not cover (for example degenerate or repeated edges) are left open — reason from the principle above.

Phase 1 Objectives

- Apply structural signal-based scoring
- Maintain consistent streaming graph state under Phase 1 rules

Phase 1 Constraints Checklist

These cover how your service runs and responds. The scoring model is described above.

Constraint	Expectation
Score range	$0.0 \leq \text{riskScore} \leq 1.0$
Lookback	Active history is the most recent 24 hours
Batch processing	Process request array in order ; preserve response order
Idempotency	Duplicate <code>txId</code> → original score, no state mutation
Missing optionals	Absent <code>ipAddress</code> / <code>deviceId</code> must not cause failure
Unknown fields	Ignore gracefully
Reset	Must fully clear graph / derived state

Later phases introduce additional signals; behaviours not covered here are left for you to reason about.

Phase 1 Examples

These examples show transaction sequences from first to last. Assume that the preceding transactions have already been scored, and that the final transaction is now being evaluated.

Example 1 - Isolated

1. Meridian Holdings → Apex Logistics

Interpretation:

- A single entity-to-entity transaction has occurred. No network pattern has emerged yet.

Example 2 - Extension

1. Meridian Holdings → Apex Logistics
2. Apex Logistics → Cascade Payments

Interpretation:

- Funds move onward from Apex Logistics to a new counterparty. The network is growing in a single direction along a plausible commercial payment chain.

Example 3 - Convergence

1. Meridian Holdings → Apex Logistics
2. Meridian Holdings → Horizon Capital
3. Apex Logistics → Sterling Bridge
4. Horizon Capital → Sterling Bridge

Interpretation:

- Two separate paths from Meridian Holdings arrive at the same destination. Sterling Bridge is now reachable from Meridian Holdings via more than one route.
- Convergence is an intermediate structural signal: stronger than simple extension, but not necessarily as suspicious as a return path.

Example 4 - Return

1. Meridian Holdings → Apex Logistics
2. Apex Logistics → Cascade Payments
3. Cascade Payments → Oakridge Imports
4. Oakridge Imports → Apex Logistics

Interpretation:

- Oakridge Imports sends funds back to Apex Logistics — a counterparty that earlier sat upstream of Oakridge Imports. Money has begun moving back toward an earlier part of the network.

Example 5 - Multi-Loop

1. Meridian Holdings → Apex Logistics
2. Apex Logistics → Cascade Payments
3. Cascade Payments → Meridian Holdings
4. Apex Logistics → Nimbus Trading
5. Nimbus Trading → Meridian Holdings

Interpretation:

- Meridian Holdings receives funds via two separate return routes through the same network. Multiple flows have converged back toward the origin.

Expected Ordering

These examples are intended to illustrate increasing structural signal. For the last transaction in each example:

- Example 1 should receive the lowest risk score of the five.
- Example 4 should receive a meaningfully higher risk score than Example 2.
- Example 5 should receive a meaningfully higher risk score than Example 4. Two independent return paths converging on the same node represent a stronger structural signal than a single return.

Phase 1 Diagnostics Vocabulary

Phase 1 evaluations can emit the following observation categories:

- STRUCTURAL_DEVIATION : Disagreement detected in the evaluation of structural signals.
- TEMPORAL_DEVIATION : Disagreement detected in temporal signal evaluation or lookback window handling.

Later Phases (locked)

Phase	Theme	Status
Phase 2	Shared devices / IPs (identity signal)	Unlocked during the event
Phase 3	Amount trails along flows (value signal)	Unlocked during the event

Design your service so optional fields can be ignored today and used tomorrow. Official Phase 2 and Phase 3 documents will be released when those phases start.